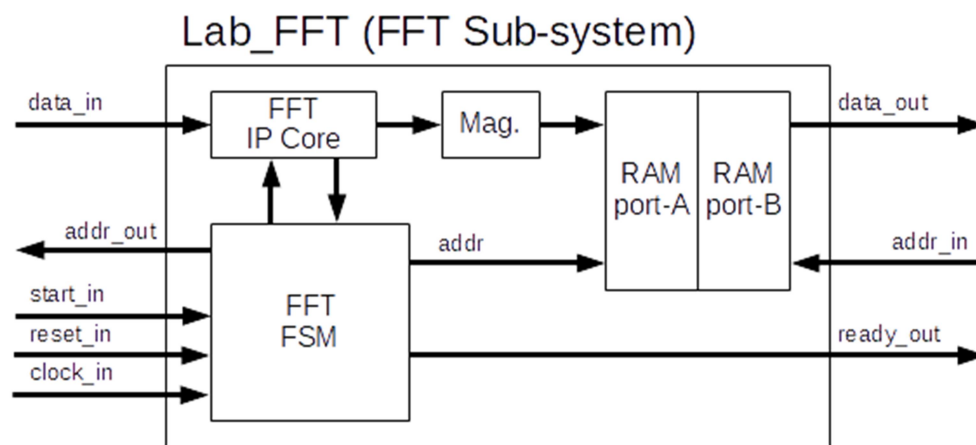


EEE 491 Lab Assignment

Lab-FFT: Fast Fourier Transform

1. Technical Specifications

Lab-FFT shall read 512 samples of 20-bit data from the Lab-WINDOW's dual-port RAM and then execute a 512-point FFT function. Squared magnitude from FFT real part output (30-bit) and FFT imaginary part (30-bit) shall be calculated and then truncated down to 32-bit and then shall be stored (written) in a simple dual-port RAM. The size of the RAM shall be 256 words to keep results up to the half of the sampling frequency. Use "Fast Fourier Transform", "Multiplier", and "Block Memory generator" building blocks from the Vivado "IP Catalog".



Lab-FFT in FPGA shall have the following input/output (IO) signals.

- "reset_in" input control signal, which resets all the registers in the design.
- "clock_in" input clock signal of the 100MHz clock oscillator on the Basys-3 board.
- "addr_out" output signal that addresses the read-only port of the dual-port RAM in Lab-WINDOW.
- "data_in" input signal from the read-only port of the dual-port RAM in Lab-WINDOW.
- "addr_in" input signal that addresses the read-only port of the dual-port RAM in Lab-FFT.
- "data_out" input signal of the read-only port of the dual-port RAM in Lab-FFT.
- "start_in" input control signal, which starts the FFT function for all 512 samples. This signal is asserted high during a single clock pulse (refer to waveform in Lab-ADC assignment).
- "ready_out" output status signal that indicates if FFT function is completed over the 512 samples and output is written to dual-port RAM of Lab-FFT and the function is ready for the next execution. This signal is active high and asserted low just after the start signal is asserted (refer to waveform in Lab-ADC assignment).

Write a test-bench in VHDL for the simulation of your design. Verify by using your test-bench simulation results that your design satisfies the technical specifications.

Integrate Lab-FFT with the other available sub-systems whenever possible in a new top level design by using "component" statements in VHDL. The components may be Lab-CTRL, Lab-PCB, ADC board, Lab-ADC, Lab-WINDOW, and Lab-DEBUG.

Lab-DEBUG accesses the read-only Port-B of the dual-port RAM of Lab-FFT and transfers the data to MATLAB. Control and observe the process sequence using start_in and ready_out signals of the components. On the Basys-3 board, use a button for each start_in signaling and a LED for each ready_out signals of the components.

2. Demonstration

- Show that your design is implemented on FPGA.
- Present your test bench simulation results: inputs, outputs (waveforms). FFT results at the dual-port RAM output must be shown clearly on the waveforms. Show that the results satisfy the requirements.
- Configure your integrated design on Basys-3 board. Use start_in button and ready_out LEDs of the components to observe/verify process sequence. If you need you may add more LEDs for further observations
- Run the integrated system (with ADC module, Lab-ADC, Lab-WINDOW, Lab-FFT) and transfer the FFT output data of each frame to MATLAB, plot and verify the FFT output results of frequency versus log of magnitude on the MATLAB.
- Use signal generator and sample various sinusoidal signals at different frequencies (100Hz, 300Hz, 1KHz, 3KHz, 6KHz) and amplitudes (0.1V, 1V, 2V, 3V). Transfer the all the samples in the dual-port RAM to MATLAB by using the Lab-DEBUG. Plot FFT results, measure and verify the results for the applied sinus input frequencies.

3. Guidance

- It is important to note in advance that the input data must be in **twos-complement** representation.
- Note that the output of the FFT block has 512 words. The last 512-th data corresponds to the FFT result at the sampling frequency. You would need the data up to half of the sampling frequency, which corresponds to the 256-th data. The other half is the mirror of the data up to half of the sampling frequency. Hence, the size of the dual port RAM to store the data shall be 256.
- Read briefly the documentation of the FFT IP core, which is available by Vivado IP core GUI. In addition, pg109-xfft.pdf document is provided in Moodle, but check the revision.
- In order to add an FFT IP core from Vivado Design Suite, find the "Fast Fourier Transform" from the IP catalog, under Digital Signal Processing -> Transforms -> FFTs. In the Customize IP FFT window, you can configure the FFT IP block in several tabs. Typical selections are shown below.
- Set the number channels (one channel), transform length (512-point FFT), Clock frequency (100MHz of Basys-3 board), and simple architecture (less resources) as Radix-2 Lite with Burst I/O data flow.
- Vivado IP generator generates VHDL code and test-bench for the IP block. You can use them.

Fast Fourier Transform (9.1)

[Documentation](#) [IP Location](#) [Switch to Defaults](#)

IP Symbol

Implementation Details

Latency

☐ Show disabled ports

Component Name

xfft_0

Configuration

Implementation

Detailed Implementation

Number of Channels

1

Transform Length

512

Architecture Configuration

Target Clock Frequency (MHz)

100

[1 - 1000]

Target Data Throughput (MSPS)

50

[1 - 1000]

Architecture Choice

☐ Automatically Select
 ☐ Pipelined, Streaming I/O
 ☐ Radix-4, Burst I/O
 ☐ Radix-2, Burst I/O
 ☒ Radix-2 Lite, Burst I/O

☐ Run Time Configurable Transform Length

- Note that target data throughput has no effect for Radix-2 Lite, Burst I/O architecture.
- Data format is fixed point with no scaling (unscaled). Select rounding mode as truncation. Input data width is 20 bits and it is a vector of 512 complex values represented as twos-complement numbers. Example; 1111 1011 0101 0111 0010 (-19086). Note that the data input also contains 20-bit imaginary part; set them to zero.
- Phase factor width specifies resolution for the phase factor signals in the FFT block. You may set to 16-bits or more for more precision. Note that it consumes DSPs and Block RAMS in the FPGA.
- Activate the reset input signal to reset the IP block.
- Throttle scheme is related with the handshake with the IP block. Real time is selected which means that handshaking is simple but data must be provided continuously without wait states.

IP Symbol

Implementation Details

Latency

☐ Show disabled ports

+

S_AXIS_DATA

+

S_AXIS_CONFIG

ack

aresetn

M_AXIS_DATA

+

event_frame_started

event_tlast_unexpected

event_tlast_missing

event_data_in_channel_halt

Component Name

xfft_0

Configuration

Implementation

Detailed Implementation

Data Format

Fixed Point

Scaling Options

Unscaled

Rounding Modes

Truncation

Precision Options

Input Data Width

20

Phase Factor Width

16

Control Signals

☐ ACLKEN
☒ ARESETn (active low)

ARESETn must be asserted for a minimum of 2 cycles

Output Ordering Options

Output Ordering

Natural Order

☐ Cyclic Prefix Insertion

Optional Output Fields

☐ XK_INDEX
☐ OVFL0

Throttle Scheme

☐ Non Real Time
☒ Real Time

- Since FPGA has many block RAMS, let us use it. Otherwise, lots of resource from logic may be wasted.

The screenshot displays the Xilinx Vivado IP configuration interface for the `xfft_0` component. The left-hand side shows the IP Symbol and Implementation Details tabs. The right-hand side shows the Configuration tab, which is divided into several sections:

- Memory Options:**
 - Data:** Radio buttons for Block RAM (selected) and Distributed RAM.
 - Phase Factors:** Radio buttons for Block RAM (selected) and Distributed RAM.
 - Number of stages using Block RAM for Data and Phase Factors:** A dropdown menu set to 0.
- Reorder Buffer:**
 - Radio buttons for Block RAM (selected) and Distributed RAM.
 - ☐ Optimize Block RAM Count Using Hybrid Memories.
- Optimize Options:**
 - Complex Multipliers:** A dropdown menu set to "Use 3-multiplier structure (resource optimization)".
 - Butterfly Arithmetic:** A dropdown menu set to "Use CLB Logic".

- Configuration is done. Now, check the implementation details at the left-hand side of the window. The IP block will be in Radix-2 lite and Burst I/O structure. This means that 512 input data for the FFT operation shall be fed one-by-one serially. Resource estimates part shows the number of DSP48 Slices and Block RAMs required in the FPGA. Check that the FPGA on Basys-3 board has these resources.
- The `S_AXIS_DATA` data input port inputs 512 input data one-by-one serially, and it is a vector of 512 complex values represented as **twos-complement numbers**. The input data width comes out to be 44 bits. The first 20-bit input data (from bit-0 to bit-19) is for the real part. The second 20-bit input data (from bit-24 to bit-43) is for the imaginary part. But since you have only the real part, you should provide 20-bit zero for the imaginary data input. The FFT core accepts complex data samples, but can perform a transform on real-valued data by setting all imaginary input samples to zero.
- The `S_CONFIG` data needs only a single bit (bit-0), which sets FFT as forward or inverse FFT.
- The `M_AXIS_DATA_TDATA` output port feeds 512 output data one-by-one serially, and it is a vector of 512 complex values represented as **twos-complement numbers**. The output data width comes out to be 62 bits and part of it is usefull. For each part width (for unscaled case) calculated as "input data width + log2 (point size) + 1" which is $20 + \log_2(512) + 1 = 30$ bits. 30-bit data output (from bit-0 to bit-29) for real part and 30-bit data output (from bit-32 to bit-61) for imaginary part. Note that there are two unused bits (bit-30 and bit-31).

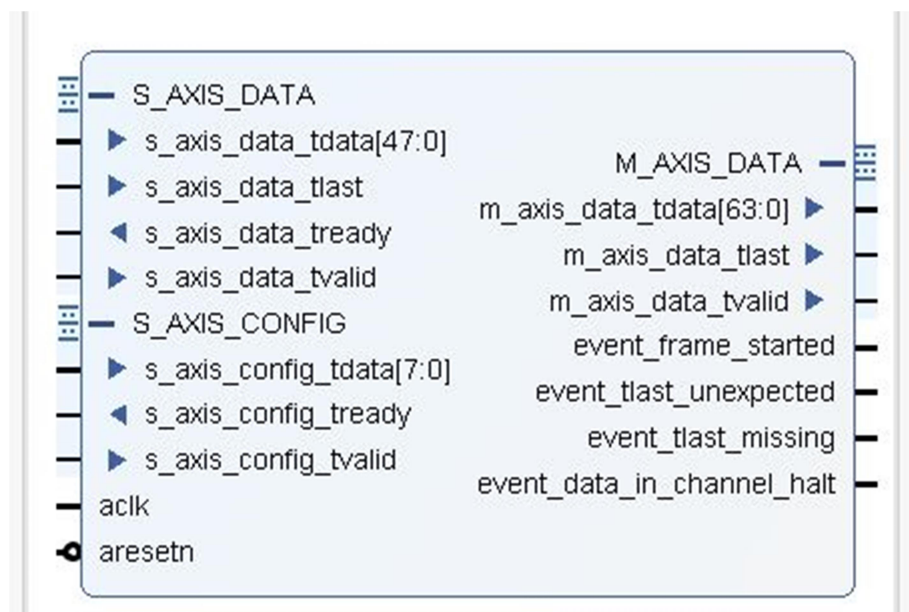
IP Symbol	Implementation Details	Latency
Information		
implementation :	Radix-2 Lite, Burst I/O	
Transform Size		
Largest :	512	
Smallest :	512	
Output Data Width :	30	
Resource Estimates Group		
DSP48 Slices :	4	
Block RAMs :	3	
AXI4 Stream Port Structure		
S_AXIS_DATA - TDATA		
<u>Transaction</u>	<u>Field</u>	<u>Type</u>
0	CHAN_0_XN_IM_0(43:24)	fix20_19
	CHAN_0_XN_RE_0(19:0)	fix20_19
1	CHAN_0_XN_IM_1(43:24)	fix20_19
	CHAN_0_XN_RE_1(19:0)	fix20_19
2	CHAN_0_XN_IM_2(43:24)	fix20_19
	CHAN_0_XN_RE_2(19:0)	fix20_19
3	CHAN_0_XN_IM_3(43:24)	fix20_19
	CHAN_0_XN_RE_3(19:0)	fix20_19
...		

IP Symbol	Implementation Details	Latency
	CHAN_0_XN_RE_2(19:0)	fix20_19
3	CHAN_0_XN_IM_3(43:24)	fix20_19
	CHAN_0_XN_RE_3(19:0)	fix20_19
...		
511	CHAN_0_XN_IM_511(43:24)	fix20_19
	CHAN_0_XN_RE_511(19:0)	fix20_19
S_AXIS_CONFIG - TDATA		
<u>Transaction</u>	<u>Field</u>	<u>Type</u>
0	FWD_INV_0(0:0)	bit1
M_AXIS_DATA - TDATA		
<u>Transaction</u>	<u>Field</u>	<u>Type</u>
0	CHAN_0_XN_IM_0(61:32)	fix30_19
	CHAN_0_XN_RE_0(29:0)	fix30_19
1	CHAN_0_XN_IM_1(61:32)	fix30_19
	CHAN_0_XN_RE_1(29:0)	fix30_19
2	CHAN_0_XN_IM_2(61:32)	fix30_19
	CHAN_0_XN_RE_2(29:0)	fix30_19
3	CHAN_0_XN_IM_3(61:32)	fix30_19
	CHAN_0_XN_RE_3(29:0)	fix30_19
...		
511	CHAN_0_XN_IM_511(61:32)	fix30_19
	CHAN_0_XN_RE_511(29:0)	fix30_19

- Check the latency for the first valid data that appears at the output of the FFT IP block. It appears to be 5663 clock cycles. However, IP block provides a “data valid” signal and “last data” output signal as the status signals (see FFT IP block symbol below). And for 100MHz clock the latency becomes only 56.63 micro seconds.

IP Symbol	Implementation Details	Latency
Transform Length	Transform Cycles	Latency(μs)
512	5663	56.630

- From the symbol, notice that the input data (S_AXIS_DATA_TDATA) stream width is 48 bits. Output data (M_AXIS_DATA_TDATA) stream width is 64 bits. For the unused bits, extend by repeating the sign bit (MSB) to unused bits. The directions of the signals are shown in the IP block symbol. There are some control input and status output signals of the IP block. You should read the document of the IP block in order to apply proper signal at proper time.
- S_AXIS_CONFIG port is related with the configuration of the FFT IP block. Configuration was already set during the config (except the forward/inverse FFT). From the symbol, notice that the configuration input data is 8-bits. Set bit-0 to "1" for forward FFT, and set rest as "0".



AXI4-Stream Considerations:

- Basic handshaking: Set TVALID=1 as you set the TDATA=D1 value. Then wait for TREADY before sending the TDATA=D2. Unloading a frame works in a similar manner, except that the core is the master in this case, that is TVALID is asserted by the core, and ready is asserted by the slave (receiver).

Note that in the Realtime mode, the following occurs:

1. The TREADY signal on the Data Output channel (m_axis_data_tready) is removed
2. The TREADY signal on the Status channel (m_axis_status_tready) is removed

3. The TVALID signal on the Data Input channel is ignored when the loading of a frame has begun. This means that only TVALID is asserted once (even for a single clock cycle) to start to load data while core can send TREADY for wait state(s). But receiver cannot assert TREADY signal to the core for wait state(s).

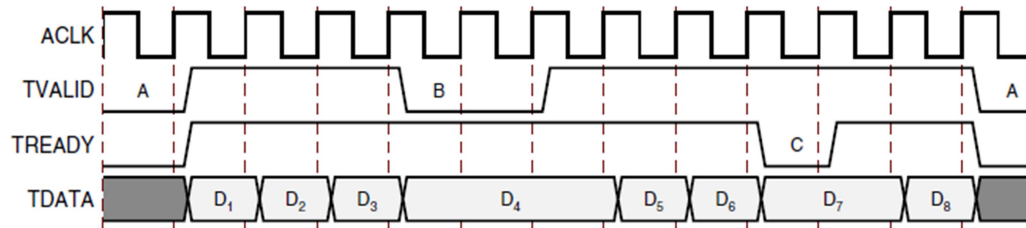


Figure 3-40: AXI Transfers and Terminology

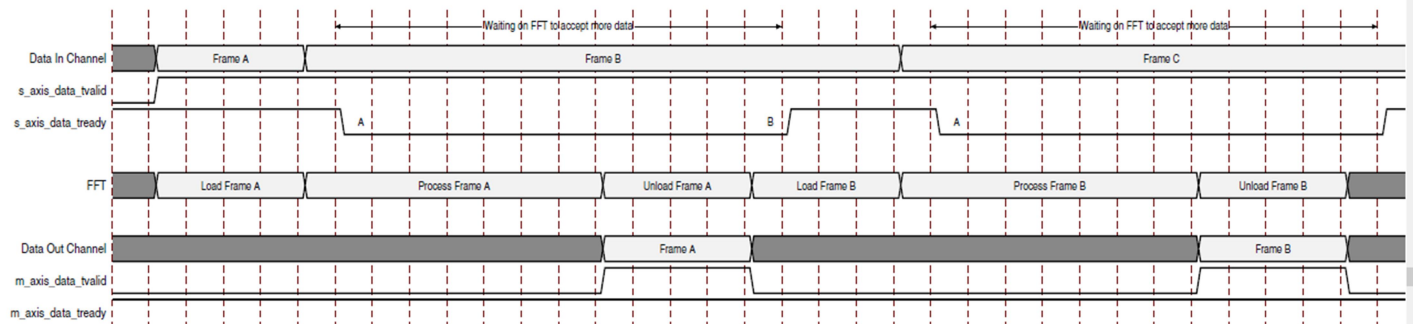


Figure 3-44: Transform Timing for Entire Frames in Burst I/O Mode with Natural Ordered Outputs

- In the above timing waveform, each frame contains 512 data. Note that FFT IP core becomes busy for a long time which is indicated by S_AXIS_DATA_TREADY=0 (from A to B). At point A in the waveform, the buffer in the Data Input channel fills, because the FFT is processing frame A and no longer draining the buffer. This can be seen externally as s_axis_data_tready going Low. The Data Input channel remains in a slave wait state situation, where the FFT cannot accept data from the upstream Master, until point B. Now the FFT has unloaded frame A and started loading Frame B into the processing core. This drains the buffer in the Data Input channel, which unblocks the Upstream Master and allows it to send the remaining data for Frame B. The situation then repeats itself with Frame C.
- Simulation example: The test bench has a ROM that contains sampled and windowed test signal (e.g. sinusoid with two frequencies) values generated on MATLAB and converted to a COE file. Lab-FFT reads the ROM and takes FFT of 512 samples. Then test bench reads data from the dual-port RAM of Lab-FFT and writes the data to a text file. Finally, MATLAB reads the output text file to compare the two FFT results for the test signal and verify.
- You may generate a text file from MATLAB for a windowed sinusoid signal samples, which is read by test-bench module for the input by using TEXTIO library, as shown below.

Library IEEE;
USE STD.TEXTIO.ALL;

The input text file must be at the same folder with the written location in the test bench code. The test-bench code should write the output of the lab-FFT design output into another text file (output.txt). Example part of a test bench VHDL code for the TEXTIO operations:

```
txt_read: process(clock_16khz)
  file readFile : TEXT open READ_MODE is "C:\Users\whereverthefileis\input.txt";
  variable myLine : LINE;
  variable val : integer;
  begin
    if rising_edge(clock_16khz) then
      if start_sampling = '1' then
```



```

        readline(readFile, myLine);
        read(MyLine,val);
        data_in <= std_logic_vector(to_signed(val,20)); -- assumed here 20-bit DATA width.
    else
        data_in <= (others=>'0');
    end if;
end if;
end process;

txt_write:process(clock_16khz)
-- Please change the path of output.txt
file writeFile : TEXT open WRITE_MODE is "C:\Users\ whereeverthefileis \output.txt";
variable sample : LINE;
begin
    if rising_edge(clock_16khz) then
        if data_enable='1' then
            write(Sample, to_integer(signed(data_out)));
            writeline(writeFile, sample);
        end if;
    end if;
end process;

```